Database Connection Pooling Every connection to a relational database carries real cost. Opening a TCP socket, completing authentication, and negotiating a session on the server side takes tens of milliseconds, and doing that for every query would make an application unusably slow under any real load. Connection pooling solves this by keeping a set of already established connections open and handing them out to application threads on demand, then returning them to the pool instead of closing them when the work is done. HikariCP is the default connection pool in Spring Boot applications and is widely regarded as the fastest and most reliable option on the JVM. It achieves its speed through aggressive bytecode-level optimization, minimal locking, and by avoiding unnecessary wrapper allocations around the underlying JDBC driver objects. A HikariCP pool is configured primarily through a small number of properties: maximum pool size, minimum idle connections, connection timeout, and max lifetime. Sizing a connection pool correctly is less intuitive than it looks. A common mistake is assuming that a larger pool always improves throughput. In practice, once the pool size exceeds the number of CPU cores available to the database server, additional connections mostly increase contention rather than parallelism, because the database itself becomes the bottleneck. A widely cited formula for sizing is connections equal to core count times two, plus the number of effective spindles, though this should always be validated against real load testing rather than applied blindly. Pool exhaustion is one of the most common production incidents involving connection pools. It happens when every connection in the pool is checked out and none are returned quickly enough, so new requests queue up waiting for a connection timeout. The usual causes are long running transactions holding a connection open, connections leaked because a try-with-resources block was not used, or a downstream dependency slowing down every query enough that the pool cannot keep up with demand. Monitoring pool utilization, wait time, and connection lifetime is essential to catching this before it becomes an outage. Max lifetime deserves particular attention. Database servers, load balancers, and firewalls frequently close idle or long lived connections without warning the client. Setting max lifetime slightly shorter than any such server side timeout ensures the pool proactively retires connections before they are killed out from under an in flight query, which otherwise surfaces as a confusing and intermittent connection reset error deep inside application code.